

Code Explanation

Testing Batch Pipeline Code

The provided code is a test suite written in Python using the unittest framework to validate the functionality of a batch data processing pipeline built with Apache Spark. The pipeline is designed to clean customer and order data by removing nulls and duplicates, and calculating the total amount for each order.

The test class `TestBatchPipeline` contains methods to set up and tear down a Spark session, as well as a test method `test\_clean\_data` to verify the correctness of the `clean\_data` function.

Overview of the Code

This code is a unit test for a batch data processing pipeline. It creates a Spark session, generates sample customer and order data, cleans the data using the `clean\_data` function, and asserts that the cleaned data meets the expected conditions.

Step 1: Setting Up the Test Environment

This step involves importing necessary modules, adding the src directory to the system path, and creating a test class `TestBatchPipeline` that inherits from `unittest.TestCase`.

```
import unittest
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
import sys
import os
sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), '../src')))
from batch_pipeline import clean_data
```

Step 2: Creating a Spark Session for Testing

The `setUpClass` method is used to create a Spark session before running any tests. This Spark session is used to create DataFrames for testing.

```
@classmethod
def setUpClass(cls):
    """Create a Spark session for testing."""
    cls.spark = SparkSession.builder
        .appName("TestCustomerOrderProcessing")
        .master("local[*]")
        .getOrCreate()
```

Step 3: Stopping the Spark Session After Testing

The `tearDownClass` method is used to stop the Spark session after all tests have been executed.

```
@classmethod
def tearDownClass(cls):
    """Stop the Spark session."""
    cls.spark.stop()
```

Step 4: Testing the clean_data Function

The `test\_clean\_data` method tests the `clean\_data` function by creating sample customer and order data, cleaning the data, and asserting that the cleaned data meets the expected conditions.

```
def test_clean_data(self):
    """Test the clean_data function."""
    customer_data = [
        (1, "John Doe", "john@example.com", "North"),
        (2, "Jane Smith", "jane@example.com", "South"),
        (3, None, "bob@example.com", "East"),
        (4, "Alice Brown", None, "West"),
        (1, "John Doe", "john@example.com", "North") # Duplicate
    ]
    customer_schema = ["CustId", "Name", "EmailId", "Region"]
    customer_df = self.spark.createDataFrame(customer_data, customer_schema)

    order_data = [
        (101, "Item1", 10.0, 2, "2023-01-01", 1),
        (102, "Item2", 15.0, 1, "2023-01-02", 2),
        (103, None, 20.0, 3, "2023-01-03", 3),
        (104, "Item4", None, 2, "2023-01-04", 4),
        (101, "Item1", 10.0, 2, "2023-01-01", 1) # Duplicate
    ]
    order_schema = ["OrderId", "ItemName", "PricePerUnit", "Qty", "Date", "CustId"]
    order_df = self.spark.createDataFrame(order_data, order_schema)

    clean_customer_df, clean_order_df = clean_data(customer_df, order_df)
```

Step 5: Verifying the Cleaned Data

This step involves asserting that the cleaned customer and order data meet the expected conditions, such as having the correct count and total amount.

```
self.assertEqual(clean_customer_df.count(), 2)
self.assertEqual(clean_order_df.count(), 2)

order_with_total = clean_order_df.filter(F.col("OrderId") == 101).first()
self.assertEqual(order_with_total["TotalAmount"], 20.0) # 10.0 * 2

order_with_total = clean_order_df.filter(F.col("OrderId") == 102).first()
self.assertEqual(order_with_total["TotalAmount"], 15.0) # 15.0 * 1
```

Step 6: Running the Unit Tests

The `unittest.main()` function is used to run the unit tests.

```
if __name__ == "__main__":
    unittest.main()
```

